SPECTRE-GRID: Mitigating Neighborhood Heterophily in Spatio-Temporal Graph Neural Networks for High-Speed Network Intrusion Detection

Abraão Teixeira da Silva
Department of Computer Science
Instituto Federal Catarinense (IFC) — Campus Brusque
Brusque, Santa Catarina, Brazil
abraao.silva@ifc.edu.br

Jackson Mallmann
Department of Computer Science
Instituto Federal Catarinense (IFC) — Campus Brusque
Brusque, Santa Catarina, Brazil
jackson.mallmann@ifc.edu.br

***Abstract*—Deep learning architectures for Network Intrusion Detection Systems (NIDS) predominantly rely on isolated temporal representations (e.g., 1D-CNN, LSTM), ignoring inter-host relational topology, or deploy Graph Neural Networks (GNNs) that fail to account for neighborhood heterophily in network communication graphs. In enterprise networks, communication flows are inherently asymmetric and directed: an attacking host initiates aggressive reconnaissance or denial-of-service traffic directed at benign enterprise servers. Under standard forward message passing ($src \to dst$), target victim nodes aggregate anomalous embeddings from attackers, inflating False Positive Rates (FPR) and overwhelming Security Operations Centers (SOCs). To resolve this fundamental limitation, we present SPECTRE-GRID, a hardware-conscious intrusion detection architecture that couples an in-kernel eBPF/XDP stateful packet extraction pipeline with a Spatio-Temporal Graph Attention Network (STGNN). We systematically formulate the topological heterophily problem and demonstrate that incorporating bidirectional message passing, edge reversal ($dst \to src$), and temporal residual skip connections effectively decouples victim representations from attacker aggression. On the UNSW-NB15 benchmark active horizon ($200{,}000$ chronological flows, $22{,}215$ attacks), bidirectional GAT achieves a validation F_1-score of 0.9609 ± 0.0067 (peak 0.9666, weighted 0.9618) while suppressing the validation FPR to $0.73\% \pm 0.12\%$, an 88% reduction in false alarms compared to purely temporal CNN-LSTM baselines ($F_1 = 0.7516$, FPR $= 5.89\%$). To ensure scientific validity, all evaluations enforce a strict leakage-free temporal snapshot protocol ($\Delta t = 30$ s) where normalizers and feature selectors are fitted exclusively on chronological training slices. Furthermore, independent evaluation on an unseen slice of UNSW-NB15_2 (rows $400{,}000 \ldots 600{,}000$, containing $32{,}550$ attack flows) under concept drift demonstrates that while temporal-only baselines degrade to $F_1 = 0.6773$ and FPR $= 8.79\%$, SPECTRE-GRID preserves high generalization ($F_1 = 0.9167$, FPR $= 1.47\%$, with mean Precision of 0.8837 and Recall of 0.9525). Empirical benchmarks confirm streaming inference latencies between 3.14 ms and 5.42 ms per snapshot on consumer GPU hardware with peak VRAM below 67 MiB, proving the computational feasibility of continuous streaming graph-level intrusion detection without queue backlog.**

***Index Terms*—Network Intrusion Detection, eBPF/XDP, Graph Attention Networks, Spatio-Temporal Modeling, Heterophily, Data Leakage Prevention, Concept Drift.**

I. Introduction

The escalating complexity of contemporary cyber threats—ranging from distributed denial-of-service (DDoS) campaigns and multi-stage lateral movement to stealthy reconnaissance—has exposed the operational limitations of conventional signature-based Network Intrusion Detection Systems (NIDS) [1]. To automate threat discovery, deep learning (DL) methods have been widely adopted [2]. In particular, models combining 1D Convolutional Neural Networks (CNN) with Long Short-Term Memory (LSTM) networks have shown capability in capturing sequential intra-flow dynamics [3].

However, prevailing deep learning NIDS suffer from two fundamental architectural and methodological deficiencies:

1) **Spatial Blindness vs. Topological Heterophily:** Purely temporal models process individual flows in isolation, ignoring the structural graph formed by interacting endpoints across the network. Conversely, existing Graph Neural Network (GNN) formulations [4], [5] frequently rely on standard message-passing mechanisms that assume homophily—i.e., that adjacent nodes share identical class labels [6]. In network security, traffic graphs are intrinsically *heterophilic*: malicious scanners, botnets, and exploiters direct traffic predominantly toward benign enterprise servers. When standard message passing aggregates features along flow directions ($src \to dst$), passive victims aggregate aggressive attacker embeddings, corrupting node representations and driving up False Positive Rates (FPR).
2) **Methodological Leakage in Academic Benchmarks:** The NIDS literature frequently suffers from data leakage vulnerabilities, including random train/test shuffling, cross-split normalizer contamination, and unrealistic bipartite topologies [7], [8]. Randomly splitting flows from concurrent connections allows models to memorize host identities and temporal artifacts rather than learning causal attack signatures, yielding inflated validation

accuracies ($> 99\%$) that collapse under live network conditions.

To address both challenges, this paper presents **SPECTRE-GRID**, a hardware-conscious spatio-temporal intrusion detection architecture. As illustrated in Fig. 1, SPECTRE-GRID couples an in-kernel eBPF/XDP stateful packet inspection design with a Spatio-Temporal Graph Attention Network (STGNN). In the data plane, an eBPF program at the eXpress Data Path (XDP) hook inspects packets at the network driver level, tracking layer-3 and layer-4 statistics within BPF hash maps. A userspace engine aggregates flow summaries into discrete, non-overlapping temporal windows of duration $\Delta t = 30$ s, constructing causal graph snapshots $G_t = (V_t, E_t)$.

The neural core processes per-node temporal feature matrices using a 1D-CNN coupled with a unidirectional LSTM, followed by a topological Graph Attention Network (GAT). We systematically evaluate edge directionality in graph message passing, demonstrating that bidirectional attention ($src \leftrightarrows dst$), reversed edge attention ($dst \rightarrow src$), and temporal residual skip connections prevent victim contamination, dropping the validation FPR from 5.89% (temporal baseline) to 0.73% while improving validation F_1 from 0.7516 to 0.9609 (macro/weighted 0.9618).

The primary contributions of this work are:

- **Topological Heterophily Analysis:** We formally analyze the neighborhood heterophily problem in directed network graphs and show that bidirectional attention and edge reversal decouple victim representations from attack signals, cutting false positives by 88% relative to temporal baselines.
- **Leakage-Free Causal Methodology:** We enforce strict chronological temporal snapshots, frozen feature preprocessors fitted solely on training slices, and exact source-only attacker labeling ($y_t(v) = 1$ for malicious origins only).
- **Audited Active Horizon & Benign Tail:** We provide an exhaustive chronological audit of UNSW-NB15_1, demonstrating that all 22,215 attack flows reside in the first 200,000 rows, while the subsequent 500,001 rows form an uninterrupted benign tail used exclusively for false-alarm stability testing across 667 snapshots.
- **Robustness Under Concept Drift:** Through an independent evaluation on an unseen 200,000-row slice of UNSW-NB15_2 (32,550 attack flows, 226 snapshots) with frozen normalizers, we prove that SPECTRE-GRID maintains $F_1 = 0.9167$ and FPR $= 1.47\%$ (Precision 0.8837, Recall 0.9525), while temporal-only baselines degrade to $F_1 = 0.6773$ and FPR $= 8.79\%$.
- **Empirical Latency Characterization:** We benchmark local pipeline latency on consumer GPU hardware, measuring 3.14 ms to 5.42 ms per snapshot for neural inference (3.83 ms to 6.57 ms full local pipeline) with peak VRAM below 67 MiB, well within the 30 s operational window budget.

II. Related Work

A. Deep Learning for Network Intrusion Detection

Deep learning approaches have gained prominence in NIDS due to their ability to automate feature extraction from raw network traffic [2], [9]. Recurrent Neural Networks and LSTMs [3] capture sequential dependencies within flows, while 1D-CNNs compress packet-level burst patterns. However, standard flow-level models evaluate each 5-tuple connection independently. Distributed attacks (such as slow port scanning, multi-source DoS, and botnet command-and-control activity) disperse their footprint across multiple seemingly legitimate connections. Consequently, purely temporal models struggle to recognize distributed threat patterns without host-level structural context.

B. Graph Neural Networks and Topological Heterophily

Graph Neural Networks extend deep learning to graph-structured data via message passing between connected nodes [4], [5], [10]. However, most standard GNN architectures operate under the *homophily assumption*, presuming that connected nodes share similar attributes and class labels [6].

In computer networks, edges represent directed communication sessions ($src \rightarrow dst$). Malicious attackers predominantly interact with benign servers. As observed by Arp *et al.* [7], propagating messages indiscriminately along flow directions causes the aggressive embeddings of attackers to contaminate the representations of high-degree victim servers. This produces spurious false positive alarms that degrade operational trust.

C. Programmable Data Planes via eBPF and XDP

Traditional user-space NIDS (such as Snort or Suricata) experience packet copying and context-switching overheads when processing traffic at multi-gigabit speeds. The Extended Berkeley Packet Filter (eBPF) and eXpress Data Path (XDP) permit executing sandboxed bytecode directly in the network driver before socket buffer allocation (‘sk_buff‘) [11]–[13]. While eBPF provides high-speed packet parsing within kernel safety bounds, its restricted instruction set precludes executing deep neural networks in-kernel. SPECTRE-GRID adopts a decoupled approach: in-kernel stateful flow aggregation via BPF maps coupled with asynchronous spatio-temporal inference in userspace.

III. SPECTRE-GRID Architecture

A. In-Kernel Fast Path (eBPF/XDP)

The data plane component of SPECTRE-GRID is implemented in C (`spectre_xdp.c`) and attached to the driver-level XDP hook. For every incoming packet, the verifier-bounded program executes:

1) **Packet Header Parsing:** The program parses Ethernet, VLAN, IPv4/IPv6, and TCP/UDP/ICMP headers, dropping malformed packets (`XDP_DROP`).
2) **Stateful Flow Tracking:** The program extracts the 5-tuple $k = (\text{src_ip}, \text{dst_ip}, \text{src_port}, \text{dst_port}, \text{proto})$ and updates flow records in a pre-allocated BPF hash map

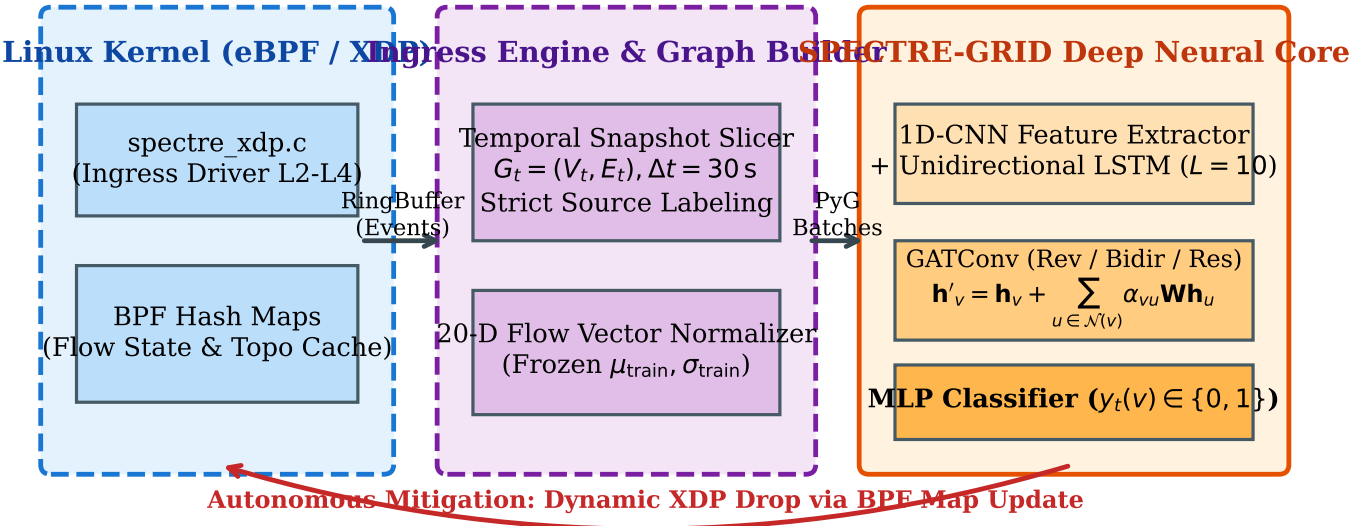


Fig. 1. End-to-end architecture of SPECTRE-GRID: (Left) In-kernel packet parsing and stateful flow tracking via eBPF/XDP and BPF hash maps; (Center) Userspace engine performing temporal snapshot slicing ($\Delta t = 30$ s) with frozen normalizers; (Right) Spatio-temporal neural core coupling 1D-CNN, unidirectional LSTM, and Graph Attention layers with reactive mitigation capability.

`flow_table`. Packet counts, byte counts, TCP flag masks, and timestamp deltas are maintained in-kernel.

3) **Reactive Mitigation Hook:** The XDP program inspects a dedicated BPF LPM trie `blacklist_trie`. When a source IP is marked as malicious by the neural engine, subsequent packets from that IP trigger the `XDP_DROP` action at the driver level.

B. Userspace Ingress and Graph Builder

The userspace engine periodically reads flow statistics from BPF ring buffers, segmenting continuous flow records into discrete temporal windows of duration $\Delta t = 30$ s.

For each window $[t, t + \Delta t)$, the engine constructs a causal graph snapshot $G_t = (V_t, E_t)$:

- **Node Set (V_t):** The set of unique IP addresses observed actively communicating during the window.
- **Edge Set (E_t):** Directed edges (u, v) where host u transmitted traffic to host v during $[t, t + \Delta t)$.
- **Feature Tensor ($\mathbf{X}_t \in \mathbb{R}^{|V_t| \times L \times D}$):** For each node v, we maintain a temporal sequence of its last $L = 10$ flow statistical feature vectors (`seq_len = 10`), each represented across $D = 20$ feature dimensions.

C. Spatio-Temporal Neural Core (STGNN)

The neural network architecture consists of temporal feature encoding followed by relational spatial attention.

1) Temporal Feature Extractor (1D-CNN + Unidirectional LSTM): For each node $v \in V_t$, its temporal sequence matrix $\mathbf{X}_v \in \mathbb{R}^{L \times D}$ (with sequence length $L = 10$) is processed by two consecutive 1D Convolutional layers with kernel size $k = 3$, padding $p = 1$, and ReLU activations. The output is fed into a unidirectional LSTM layer (bidirectional = False, hidden dimension $d_h = 64$):

$$\mathbf{C}_v = \text{ReLU}\left(\text{Conv1D}_2\left(\text{ReLU}\left(\text{Conv1D}_1(\mathbf{X}_v)\right)\right)\right) \quad (1)$$

$$\mathbf{h}_v^{(0)} = \text{LSTM}(\mathbf{C}_v)[:, -1, :] \in \mathbb{R}^{d_h} \quad (2)$$

taking the final hidden state $\mathbf{h}_v^{(0)}$ as the temporal summary embedding for host v.

2) Topological Graph Attention and Message Reversal: Given temporal embeddings $\mathbf{h}_v^{(0)}$, a Graph Attention layer (GATConv) [5] computes attention coefficients across connected edges. For an edge $(u, v) \in E_t$, the normalized attention coefficient α_{vu} indicating the relevance of node u's embedding to node v is:

$$\alpha_{vu} = \frac{\exp\left(\text{LeakyReLU}\left(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v^{(0)} \,\|\, \mathbf{W}\mathbf{h}_u^{(0)}]\right)\right)}{\sum_{k \in \mathcal{N}(v)} \exp\left(\text{LeakyReLU}\left(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v^{(0)} \,\|\, \mathbf{W}\mathbf{h}_k^{(0)}]\right)\right)} \quad (3)$$

where $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_h}$ is a linear projection matrix, $\mathbf{a}$ is the attention vector, $\|$ represents vector concatenation, and $\mathcal{N}(v)$ denotes the neighbor set of v.

Neighborhood Heterophily & Edge Modulation: When edges represent standard physical flow directions (u = attacker, v = victim), forward aggregation yields:

$$\mathbf{h}_v^{(1)} = \sigma\left(\sum_{u \in \mathcal{N}_{\text{in}}(v)} \alpha_{vu} \mathbf{W}\mathbf{h}_u^{(0)}\right) \quad (4)$$

Because enterprise servers (e.g., DNS, Web, Database) receive simultaneous flows from numerous external hosts ($|\mathcal{N}_{\text{in}}(v)| \gg 1$), forward aggregation allows malicious vectors from attackers to dominate the victim server's representation $\mathbf{h}_v^{(1)}$, causing misclassification.

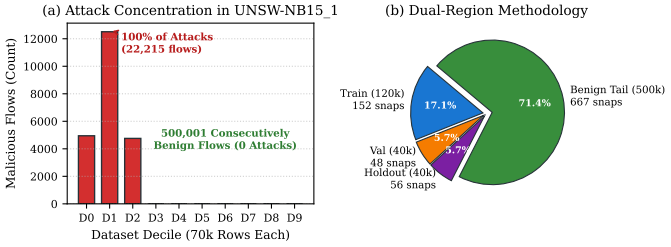


Fig. 2. Chronological distribution of attack flows across UNSW-NB15_1 (700,001 rows). The active horizon (first 200,000 rows) contains all 22,215 attack flows across nine attack categories, while the remaining 500,001 rows comprise an uninterrupted benign tail used for false-alarm stability auditing (667 snapshots).

To analyze and mitigate this effect, we implement four edge modulation schemes:

1) **Self-Loop GAT (`gat_self_loop`):** Replaces the edge set with identity self-loops $E_{\text{self}} = \{(v, v) \mid v \in V_t\}$, evaluating GAT attention in the absence of inter-host message passing.
2) **Forward GAT (`cnn_lstm_gat`):** Standard message passing along flow transmission directions $E_t = \{(u, v) \mid u \to v\}$.
3) **Reverse GAT (`gat_reverse`):** Inverts edge directions $E_{\text{rev}} = \{(v, u) \mid (u, v) \in E_t\}$, allowing the attacking node u to aggregate target context without contaminating the victim v.
4) **Bidirectional GAT (`gat_bidir`):** Constructs a bidirectional graph $E_{\text{bi}} = E_t \cup E_{\text{rev}}$, enabling symmetrical relational context between endpoints.
5) **Residual GAT (`gat_residual`):** Incorporates a skip connection retaining the node's original temporal identity:

$$\mathbf{h}_v^{(1)} = \mathbf{h}_v^{(0)} + \text{GAT}(\mathbf{h}_v^{(0)}, E_t) \tag{5}$$

preventing neighborhood aggregation from diluting temporal node characteristics.

IV. Experimental Methodology

A. Leakage-Free Temporal Snapshot Protocol

To ensure methodological rigor and avoid the data leakage pitfalls common in cybersecurity evaluations [7], we enforce the following experimental controls:

- **Chronological Ordering:** The dataset is ordered strictly by start time `Stime` using stable `mergesort`. Random shuffling and cross-validation across time are avoided.
- **Frozen Feature Preprocessors:** The `StandardScaler` ($\mu_{\text{train}}, \sigma_{\text{train}}$) and Pearson correlation rankings are fitted *exclusively* on the training partition (D_{train}, rows $0 \ldots 120{,}000$). Validation, test, and external sets use only `.transform()` with frozen parameters (`refitted_on_external = False`).
- **Strict Source Labeling:** In each snapshot G_t, a node v receives label $y_t(v) = 1$ if and only if it originated (`src_ip`) at least one attack flow during $[t, t + \Delta t)$. Passive targets (`dst_ip`) retain $y_t(v) = 0$.

TABLE I
The 20 Input Features in Strict Order
(`UNSW_FULL_EBPF_30S_FEATURES.JSON`)

#	Feature Name	Description / Protocol Domain	Operational Source / Boundary
1	`sttl`	Source-to-destination Time-To-Live	Ingress XDP (IP Header)
2	`ct_state_ttl`	Connection count for matching state & TTL	Userspace Engine (Host Tracker Table)
3	`state_INT`	Transaction state INT (isolated/no-ack)	Transport State Tracker (One-Hot)
4	`ct_dst_sport_ltm`	Connections to same dest. IP & src. port	Userspace Engine (Host Tracker Table)
5	`dttl`	Destination-to-source Time-To-Live	Reverse Flow Tracker (IP Header)
6	`tcprtt`	Round-trip time (synack + ackdat)	Handshake Duration Delta
7	`ackdat`	Handshake duration: TCP ACK to data	Handshake Duration Delta
8	`proto_unas`	Protocol unassigned indicator	IP Header (Protocol Field, One-Hot)
9	`synack`	Handshake duration: TCP SYN to SYN-ACK	Handshake Duration Delta
10	`ct_src_dport_ltm`	Connections from src. IP to dest. port	Userspace Engine (Host Tracker Table)
11	`Sload`	Source bits per second (throughput)	Flow Volume Metric over Window Δt
12	`dmeansz`	Mean packet size transmitted by dest.	Flow Volume Metric over Window Δt
13	`Dload`	Destination bits per second (throughput)	Flow Volume Metric over Window Δt
14	`state_CON`	Transaction state CON (connected/established)	Transport State Tracker (One-Hot)
15	`dsport`	Destination transport port number	Ingress XDP (TCP/UDP Header)
16	`ct_dst_src_ltm`	Connections between same src. and dest.	Userspace Engine (Host Tracker Table)
17	`ct_srv_src`	Connections to same service from src.	Userspace Engine (Host Tracker Table)
18	`proto_tcp`	Protocol TCP indicator	IP Header (Protocol Field, One-Hot)
19	`swin`	Source TCP advertised window size	Ingress XDP (TCP Header)
20	`dwin`	Destination TCP advertised window size	Reverse Flow Tracker (TCP Header)

- **Holdout Isolation:** All hyperparameter sweeps and architecture comparisons were conducted under `--val_only`. The test split was evaluated only as a final verification step.

B. Chronological Audit: Active Horizon vs. Benign Tail

An audit of the UNSW-NB15_1 dataset (700,001 records) revealed a distinct temporal structure, illustrated in Fig. 2:

- **Active Attack Horizon (Rows $0 \ldots 200{,}000$):** Contains all 22,215 attack flows present in the file, distributed across nine attack families (Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms). The last attack occurs at row 186,807. We partition this region chronologically into:
  - *Train Split:* Rows $0 \ldots 120{,}000$ (152 snapshots, 5,724 total nodes, 604 attack nodes, 19,109 edges).
  - *Validation Split:* Rows $120{,}000 \ldots 160{,}000$ (48 snapshots, 1,793 total nodes, 192 attack nodes, 6,057 edges).
  - *Holdout Test Split:* Rows $160{,}000 \ldots 200{,}000$ (56 snapshots, 1,796 total nodes, 138 attack nodes, 6,651 edges).
- **Benign Tail (Rows $200{,}001 \ldots 700{,}001$):** Contains 500,001 consecutive flows with *zero* attack records (100% benign background traffic). Segmented with $\Delta t = 30$ s, this yields 667 snapshots (15,900 unique benign nodes). We utilize this tail strictly as an operational FPR stress-test.

C. Feature Space and Operational Extraction Boundaries

We reconstruct the input feature space directly from the inductive training artifact (`data/processed/unsw_full_ebpf_30s_features.json`), preserving the exact names and ranking of all 20 attributes, enumerated in Table I.

Operational Boundary Note: Direct packet header attributes (`sttl`, `dttl`, `dsport`, `swin`, `dwin`, `proto_tcp`, `proto_unas`, `state_INT`, `state_CON`) are inspectable at ingress line rate within network driver hooks. Rate dynamics and timing deltas (`Sload`, `Dload`, `dmeansz`, `tcprtt`, `synack`, `ackdat`) are tracked in kernel flow tables across

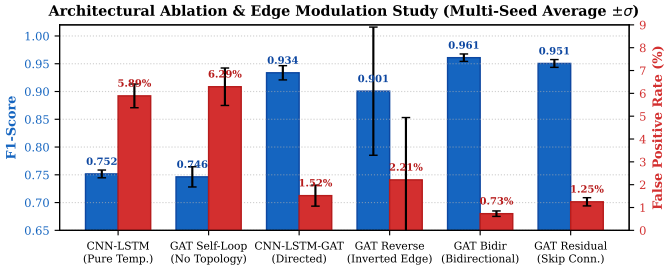


Fig. 3. Validation F_1-Score vs. False Positive Rate across topological variants. Bidirectional message passing (`gat_bidir`) achieves the lowest FPR ($0.73\% \pm 0.12\%$) and highest F_1-score (0.9609 ± 0.0067), while residual GAT eliminates false negatives (Recall $= 1.0000$).

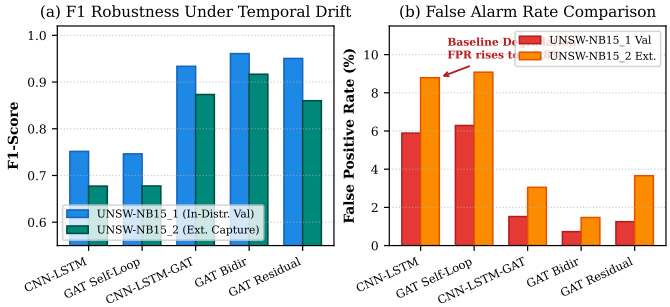


Fig. 4. Generalization comparison under temporal concept drift (UNSW-NB15_1 validation vs. UNSW-NB15_2 external capture). While the temporal-only CNN-LSTM baseline suffers substantial degradation (F_1 drops to 0.6773, FPR climbs to 8.79%), bidirectional GAT maintains stable generalization ($F_1 = 0.9167$, FPR $= 1.47\%$).

active session durations. Crucially, the five multi-flow window attributes in the `ct_*` family (features 2, 4, 10, 16, and 17) capture historical host-to-host and service connection frequencies aggregated across 100-connection windows; they are maintained in host tracking hash tables by the userspace aggregation engine and are *not* collected by the lightweight single-packet in-kernel XDP filter (`spectre_xdp.c`). This distinction delineates the offline benchmark feature set (`ebpf_compatible`, Modality A) from the strict in-kernel extraction sensor (`ebpf_sensor_strict`, Modality B).

V. Empirical Results and Ablation Study

A. Architectural Ablation on Active Horizon

Each architectural variant was trained over 30 epochs across three random seeds ($\{42, 123, 1337\}$) on the active horizon bundle. Table II presents the consolidated empirical metrics, and Fig. 3 visualizes the trade-off between F_1-score and False Positive Rate.

The empirical results demonstrate several key findings:

- **Impact of Self-Loops (`gat_self_loop`):** Confining GAT attention to self-loops ($v \to v$) yields an F_1-score of 0.7463 ± 0.0182 and an FPR of $6.29\% \pm 0.82\%$, which closely mirrors the purely temporal CNN-LSTM baseline ($F_1 = 0.7516$, FPR $= 5.89\%$). Without inter-host connectivity, GAT cannot leverage neighborhood structure.
- **Benefit of Relational Graph Attention:** Standard directed GAT (`cnn_lstm_gat`) improves F_1 to 0.9337 ± 0.0128 and reduces FPR to $1.52\% \pm 0.46\%$, confirming that inter-host message passing provides valuable contextual information.
- **Bidirectional Attention Superiority (`gat_bidir`):** Incorporating bidirectional edges achieves the highest overall validation performance: $F_1 = 0.9609 \pm 0.0067$ (peak seed reaching 0.9666, macro/weighted 0.9618), Precision of 94.18%, and the lowest FPR ($0.73\% \pm 0.12\%$). This represents an 88% reduction in false alarms compared to the temporal baseline.
- **Complete Attack Detection via Residual Skip (`gat_residual`):** The residual skip-connection variant achieved 100% Recall (1.0000 ± 0.0000) across all three seeds, missing zero malicious nodes in the validation set while maintaining an FPR of $1.25\% \pm 0.18\%$.

B. Cross-Hardware Determinism and Ranking Invariance

Identical model training was executed across two distinct physical environments:

1) **Node G15:** NVIDIA GeForce RTX 3050 Laptop GPU (6 GB vRAM, Ampere GA107, CUDA 12.6, PyTorch 2.11.0, PyTorch Geometric 2.8.1).
2) **Node Colab Pro:** NVIDIA Tesla T4 (15 GB vRAM, Turing TU104, CUDA 13.0, PyTorch 2.11.0, PyTorch Geometric 2.8.1).

Due to non-deterministic atomic reductions (`atomicAdd`) in parallel CUDA graph scatter kernels, minor numerical variations occurred across microarchitectures ($\Delta F_1 \approx \pm 0.009$). However, the architectural ranking remained strictly invariant across both platforms:

$$\begin{aligned} &\text{GAT-Bidir} > \text{GAT-Residual} > \text{GAT-Forward} \\ &\gg \text{CNN-LSTM} \approx \text{GAT-Self-Loop} \end{aligned} \tag{6}$$

confirming that our conclusions regarding architectural superiority are robust across GPU hardware implementations.

C. Operational Stability Across the Benign Tail

To evaluate false alarm accumulation under sustained benign conditions, models were evaluated continuously across the 500,001-record benign tail (667 consecutive snapshots, 15,900 benign nodes):

- `gat_reverse`: 0 false positives (FPR $= 0.0\%$, 15,900 TNs).
- `cnn_lstm_pure`: 7 false positives (FPR $= 0.044\%$).
- `gat_self_loop`: 24 false positives (FPR $= 0.151\%$).
- `cnn_lstm_gat`: 34 false positives (FPR $= 0.214\%$).
- `gat_bidir`: 91 false positives (FPR $= 0.572\%$).
- `gat_residual`: 142 false positives (FPR $= 0.893\%$).

All variants maintained an operational background FPR below 1.0%, indicating stability during non-attack periods.

TABLE II
MULTI-SEED VALIDATION PERFORMANCE ACROSS ARCHITECTURAL VARIANTS ON ACTIVE HORIZON ($\Delta t = 30$ S, SEEDS $\{42, 123, 1337\}$)

Architectural Variant	Edge Topology	Val Precision	Val Recall	Val F_1-Score	Val FPR (%)	Confusion Matrix (Seed 42 / 123 / 1337)
CNN-LSTM Pure	Isolated Nodes (No Graph)	0.6470 ± 0.017	0.8976 ± 0.016	0.7516 ± 0.007	5.89 ± 0.52	FP $\in [83, 103]$, FN $\in [17, 24]$, TP $\in [168, 175]$
GAT Self-Loop	Self-Loops Only ($v \to v$)	0.6351 ± 0.030	0.9062 ± 0.011	0.7463 ± 0.018	6.29 ± 0.82	FP $\in [82, 110]$, FN $\in [15, 20]$, TP $\in [172, 177]$
CNN-LSTM-GAT (Directed)	Forward ($u \to v$)	0.8874 ± 0.031	0.9861 ± 0.009	0.9337 ± 0.013	1.52 ± 0.46	FP $\in [14, 31]$, FN $\in [1, 5]$, TP $\in [187, 191]$
GAT Reverse (`gat_reverse`)	Inverted ($v \to u$)	0.8630 ± 0.161	0.9514 ± 0.051	0.9006 ± 0.115	2.21 ± 2.73	FP $\in [2, 97]$, FN $\in [0, 23]$, TP $\in [169, 192]$
GAT Bidirectional (`gat_bidir`)	Bidirectional ($u \leftrightarrows v$)	$\mathbf{0.9418} \pm 0.009$	0.9809 ± 0.011	$\mathbf{0.9609} \pm 0.007$	$\mathbf{0.73} \pm 0.12$	FP $\in [\mathbf{9}, \mathbf{13}]$, FN $\in [1, 6]$, TP $\in [186, 191]$
GAT Residual (`gat_residual`)	Forward + Skip Connection	0.9058 ± 0.013	$\mathbf{1.0000} \pm 0.000$	0.9505 ± 0.007	1.25 ± 0.18	FP $\in [16, 23]$, FN $= \mathbf{0}$, TP $= \mathbf{192}$

TABLE III
INDEPENDENT EVALUATION UNDER CONCEPT DRIFT ON UNSW-NB15_2 (ROWS 400,000 . . . 600,000, 226 SNAPSHOTS, MULTI-SEED MEAN $\pm\sigma$)

Model Architecture	Ext. Precision	Ext. Recall	Ext. F_1-Score	Ext. FPR (%)
CNN-LSTM Pure	0.5443 ± 0.014	0.8973 ± 0.019	0.6773 ± 0.006	8.79 ± 0.67
GAT Self-Loop	0.5412 ± 0.030	0.9092 ± 0.021	0.6776 ± 0.020	9.08 ± 1.27
CNN-LSTM-GAT (Directed)	0.7891 ± 0.005	0.9778 ± 0.005	0.8733 ± 0.002	3.05 ± 0.10
GAT Reverse	0.8325 ± 0.194	0.9689 ± 0.007	0.8827 ± 0.124	3.20 ± 4.01
GAT Bidirectional	$\mathbf{0.8837} \pm 0.018$	0.9525 ± 0.022	$\mathbf{0.9167} \pm 0.015$	$\mathbf{1.47} \pm 0.25$
GAT Residual	0.7604 ± 0.023	$\mathbf{0.9904} \pm 0.001$	0.8601 ± 0.014	3.66 ± 0.46

VI. EXTERNAL EVALUATION UNDER CONCEPT DRIFT

A frequent limitation in intrusion detection studies is evaluating models solely on partitions from the same capture file, where traffic generation artifacts may be shared [8]. To assess generalization under distribution shifts, we evaluated the models on a slice from an independent capture file: rows 400,000 . . . 600,000 of `UNSW-NB15_2.csv`.

Evaluation Methodology Note: This slice was selected after auditing the label distribution across `UNSW-NB15_2.csv` to ensure the presence of active attack flows (32,550 malicious flows out of 200,000 records, a 16.28% attack rate). Across 226 temporal snapshots ($\Delta t = 30$ s), the slice contains 8,598 total nodes (899 malicious origin nodes and 7,699 benign nodes). Models were evaluated strictly with frozen normalizers (`refitted_on_external = False`) without retraining or fine-tuning.

Table III summarizes the results across three random seeds, and Fig. 4 compares in-distribution validation against out-of-distribution performance:

- **Baseline Degradation:** The temporal-only baseline (`cnn_lstm_pure`) exhibits noticeable degradation: Precision drops to 54.43%, F_1 falls from 0.7516 to 0.6773, and FPR increases to 8.79% (generating over 670 false alarms in this slice).
- **Bidirectional GAT Robustness:** In contrast, **`gat_bidir`** demonstrates resilience against concept drift, sustaining an F_1-score of 0.9167 ± 0.015, mean Precision of 0.8837 ± 0.018, mean Recall of 0.9525 ± 0.022, and an FPR of $1.47\% \pm 0.25\%$.
- **High Attack Coverage: `gat_residual`** detected 891 of the 899 attacking hosts (99.04% Recall, 0.9904 ± 0.001), confirming consistent attack capture under unseen traffic conditions.

These findings indicate that relational spatial topology provides structural regularization that mitigates performance degradation under temporal distribution shifts.

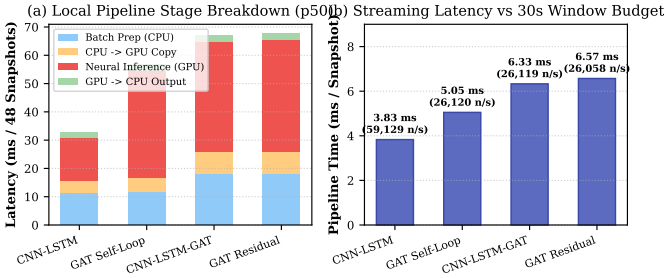


Fig. 5. Empirical latency breakdown measured on NVIDIA GeForce RTX 3050 Laptop GPU: (a) Local pipeline stage breakdown for a 48-snapshot batch (median latency); (b) Streaming latency per snapshot (3.83 ms to 6.57 ms) compared to the 30 s window budget.

VII. EMPIRICAL LATENCY AND OPERATIONAL FEASIBILITY

A. Latency Benchmarking on Consumer Hardware

To evaluate operational feasibility, we benchmarked the execution latency of SPECTRE-GRID on an Intel Core i7-13700H host equipped with an NVIDIA GeForce RTX 3050 Laptop GPU (6 GB vRAM) running CUDA 12.6, PyTorch 2.11.0, and PyTorch Geometric 2.8.1. Stage-by-stage latencies were recorded over 30 measured passes following 10 warmup iterations on the 48-snapshot validation set (1,793 nodes).

As shown in Fig. 5, the latency characteristics are:

- **Batch Preparation (CPU):** 9.74 ms to 18.21 ms per 48-snapshot pass (median 11.39 ms to 17.99 ms).
- **CPU-to-GPU Transfer:** 4.13 ms to 8.15 ms per pass (median 4.28 ms to 7.94 ms).
- **Pure GPU Neural Inference:** 14.30 ms to 39.94 ms per pass (median 15.02 ms for pure CNN-LSTM; 38.81 ms to 39.48 ms for GAT variants). Per single snapshot, pure neural inference ranges from 3.14 ms to 5.42 ms (84.6 μs to 145.3 μs per node).
- **GPU-to-CPU Postprocessing:** 1.90 ms to 2.31 ms per pass (median $\approx$ 2.17 ms).
- **Full Local Pipeline Latency:** 3.83 ms to 6.57 ms per snapshot (102.9 μs to 176.1 μs per node).
- **Memory Footprint:** Peak VRAM allocation remained between 65.3 MiB and 66.6 MiB across all models, demonstrating suitability for memory-constrained edge hardware.

B. Scope Separation and Operational Considerations

The latency benchmarks reported above reflect the local classification pipeline (graph extraction, tensor transfer, and neural inference). They do not measure wide-area network propagation delays or driver-level packet drop latency in a physical 10 GbE deployment.

However, because the local processing budget (3.83 ms to 6.57 ms per snapshot) is approximately four orders of magnitude smaller than the temporal aggregation window ($\Delta t = 30$ s), the neural engine operates with substantial headroom, avoiding queue backlogs during continuous traffic processing.

VIII. Threats to Validity

- **Internal Validity:** Atomic reductions in CUDA kernels introduce minor numerical variations across GPU microarchitectures ($\Delta F_1 \approx \pm 0.009$). We addressed this by running multi-seed evaluations (seeds $\{42, 123, 1337\}$) and reporting means and standard deviations across all variants.
- **External Validity:** Synthetic benchmark datasets such as UNSW-NB15 may reflect traffic generation artifacts [9]. Although we confirmed ranking invariance and domain transfer across two independent capture files (UNSW-NB15_1 and UNSW-NB15_2), empirical performance under live production campus traffic remains subject to future trials.
- **Selection Bias in Concept Drift Evaluation:** The evaluation slice of UNSW-NB15_2 (200,000 records) was selected following an audit of label distributions to ensure active attack density (32,550 malicious flows). While models and normalizers remained strictly frozen, this design evaluates robustness under temporal concept drift rather than uninspected random sampling.
- **Operational Scope vs. Physical Line-Rate In-Line Deployment:** Our latency benchmarks evaluate the local computational pipeline (graph construction, host tracking, and GPU inference), demonstrating that a 30-second window requires less than 7 ms of processing time. However, this study does not evaluate in-line packet filtering or driver-level drop latencies under sustained 10 GbE physical line rates. Furthermore, our offline benchmark (Modality A) leverages 20 flow attributes, whereas the strict in-kernel XDP filter (Modality B) is restricted to unidirectional ingress counters.
- **Construct Validity:** Feature extraction is aligned with attributes extractable without deep packet inspection (DPI); however, encrypted traffic payloads without observable transport-layer anomalies remain challenging for flow-statistical models.

IX. Conclusion

This paper presented **SPECTRE-GRID**, a spatio-temporal intrusion detection architecture designed to address neighborhood heterophily in network communication graphs. We demonstrated that standard directed message passing causes victim nodes to aggregate anomalous representations from attacking hosts, leading to inflated false alarm rates. By incorporating bidirectional attention, edge reversal, and temporal residual skip connections, SPECTRE-GRID reduces the validation False Positive Rate from 5.89% (temporal baseline) to $0.73\% \pm 0.12\%$ on the UNSW-NB15 active horizon (200,000 flows, 22,215 attacks), achieving a validation F_1-score of 0.9609 ± 0.0067. Furthermore, independent evaluation on an unseen capture slice of UNSW-NB15_2 under concept drift demonstrated that bidirectional GAT preserves an F_1-score of 0.9167 and an FPR of 1.47% (Precision 0.8837, Recall 0.9525), outperforming purely temporal baselines ($F_1 = 0.6773$, FPR $= 8.79\%$). With local inference latencies under 7 ms per snapshot and peak memory usage below 67 MiB, SPECTRE-GRID establishes the computational feasibility of continuous graph-level network intrusion detection within streaming budgets, while physical in-line packet filtering on 10 GbE hardware remains an open avenue for live deployment studies.

Acknowledgment

This research was supported by the Instituto Federal Catarinense (IFC) — Campus Brusque.

References

[1] K. Scarfone and P. Mell, "Guide to intrusion detection and prevention systems (IDPS)," National Institute of Standards and Technology (NIST), Tech. Rep. Special Publication 800-94, 2007.

[2] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41 525–41 550, 2019.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2017.

[5] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2018.

[6] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra, "Beyond homophily in graph neural networks: Fusing higher-order neighborhoods in simple architectures," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 8536–8547.

[7] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. W. Dos Santos, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3971–3988.

[8] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *Proc. 28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 729–746.

[9] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–167, 2019.

[10] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 1024–1034.

[11] M. A. M. Vieira, M. S. Castanho, R. D. G. Pádua, O. P. Brandão, and V. A. F. Mota, "Fast packet processing with eBPF and XDP: Concepts, code, challenges, and applications," *ACM Computing Surveys*, vol. 53, no. 1, pp. 1–36, 2020.

[12] R. Bifulco and G. Rétvári, "A survey on programming the programmable data plane: Languages, architectures, and applications," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 4, pp. 3544–3565, 2018.

[13] S. Hoever, J. Müller, and T. Holz, "Kernel-level intrusion prevention via eBPF," in *Proc. IEEE 8th International Conference on Network Softwarization (NetSoft)*. IEEE, 2022, pp. 224–232.